FIAP

Data Structure and Algorithms

Ciência da Computação

Prof. Érick T. Yamamoto

Syllabus - Variáveis e Tipos de Dados em C

Ao final da aula, os alunos deverão ser capazes de:

- entender o que é uma variável;
- identificar os principais tipos de dados em C;
- declarar e inicializar variáveis;
- armazenar e exibir valores;
- utilizar printf e scanf com os formatos corretos;
- utilizar operadores aritméticos;
- compreender operadores relacionais;
- realizar entrada e saída de dados com mais segurança;
- montar expressões em C corretamente.

Problema!

Quando um programa precisa guardar um número, um nome, uma idade ou uma nota, onde essas informações ficam?

Para guardar dados temporariamente durante a execução do programa, usamos variáveis.

O que é uma variável?

Uma variável é um **espaço na memória** usado para armazenar um valor.

Exemplo simples:

- idade de um aluno
- nota de uma prova
- altura de uma pessoa
- letra inicial de um nome

Uma boa analogia para explicar:

A variável funciona como uma caixinha com um nome, onde guardamos um valor.

O que é uma variável?

Estrutura básica de uma variável em C

```
tipo nome_da_variavel;
```

Exemplos:

```
int idade;
float altura;
char letra;
```

Se quiser já atribuir um valor:

```
int idade = 20;
float altura = 1.75;
char letra = 'A';
```

IMPORTANTE!

- **char usa aspas simples**
- **strings usam aspas duplas**

Tipo	Tamanho típico	Bits	Intervalo aproximado
`char`	1 byte	8 bits	-128 a 127
`int`	4 bytes	32 bits	~ -2 bilhões a +2 bilhões
`float`	4 bytes	32 bits	~ 6 a 7 casas decimais
`double`	8 bytes	64 bits	~ 15 casas decimais

O que é uma variável?

As variáveis podem ser declaradas de qualquer maneira?

```
int 1idade = 20;
float nota de matematica = 8.5;
char int;
```

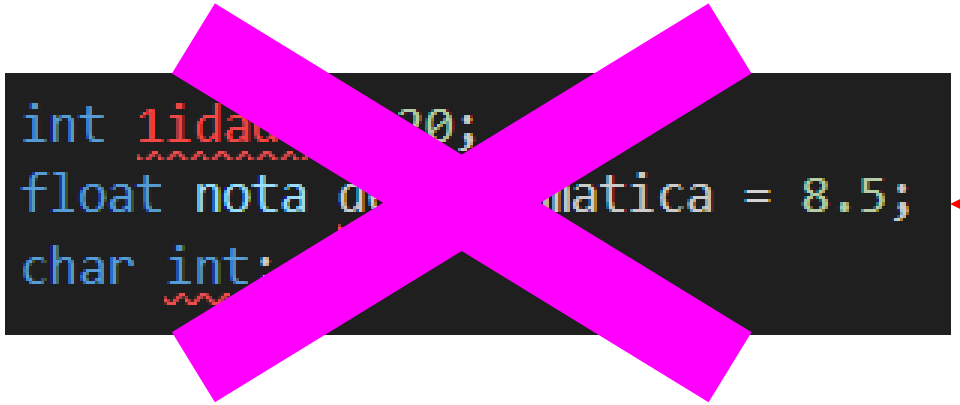
O que vocês acham dessas declarações?

Ou seja, nomes de variáveis:

- podem ter letras, números e _;
- não podem começar com número;
- não podem ter espaço;
- não podem usar palavras reservadas da linguagem.

O que é uma variável?

As variáveis podem ser declaradas de qualquer maneira?



O que vocês acham dessas declarações?

Ou seja, nomes de variáveis:

- podem ter letras, números e _;
- não podem começar com número;
- não podem ter espaço;
- não podem usar palavras reservadas da linguagem.

Exibindo variáveis com printf

Você viu nas aulas passadas que podemos exibir algo através da biblioteca,

#include<stdio.h>

```c
#include<stdio.h>

int main() {
    printf("Hello, World!\n");
    printf("Bem-vindo a Disciplina de Estruturas de Dados e Algoritmos com C\n");
    return 0;
}
```

No terminal fica:

```
Hello, World!
Bem-vindo a Disciplina de Estruturas de Dados e Algoritmos com C
```

Será que é possível exibir as variáveis?

Exibindo variáveis com printf

Sim, é possível. Através dos **especificadores de formato.**

Eles são **instruções para o printf dizendo como interpretar os bits da variável na memória.**

Porque, no fundo, tudo que o computador tem é isso:

01010101 00001111 10101010 ...

Ou seja, **bits.**

O printf precisa saber → “Esses bits representam um inteiro? Um decimal? Um caractere?”

E é isso que o **formato (%d, %f, etc.) informa**.

Exibindo variáveis com printf

Resumindo... Em uma tabela, fica assim:

Tipo	Uso	Exemplo	Formato
`int`	inteiro	10	`%d`
`float`	decimal	1.75	`%f`
`double`	decimal com maior precisão	3.14159	`%lf`
`char`	caractere	'A'	`%c`

Por que isso acontece?

A linguagem C segue o padrão internacional (principalmente dos EUA), onde:

1.75 → decimal com ponto

Já no Brasil usamos:

1,75 → decimal com vírgula

Mas **linguagens de programação usam ponto por padrão**.

No especificador de formato para float, se quisermos controlar a quantidade de casas decimais utilizamos entre % e f; Um . e a quantidade de casas. Por exemplo duas casas seria %.2f

Agora quero interagir com o código - função scanf

Para que possamos ler valores de um usuário, a biblioteca #include<stdio.h>

também tem uma função chamada **scanf**

```c
#include <stdio.h>

int main() {
    int idade;

    printf("Digite sua idade: ");
    scanf("%d", &idade);

    printf("Sua idade e: %d\n", idade);

    return 0;
}
```

print, somente para exibir

scanf, aguardando o usuário digitar

print, com especificador de formato

```
s\output> & .\'main.exe'
Digite sua idade:
```

O & indica o endereço da variável na memória, para que o scanf consiga armazenar o valor digitado.

Vamos colocar a matemática e lógica - Operadores

Se eu já sei guardar números em variáveis... como eu faço contas com eles?

Resposta: Usamos **operadores**.

Existem dois tipos de operadores → Matemático e Relacionais

Matemático		
Operador	**Significado**	**Exemplo**
+	soma	a + b
–	subtração	a - b
*	multiplicação	a * b
/	divisão	a / b
%	resto da divisão	a % b

Relacionais	
Operador	**Significado**
==	igual
!=	diferente
>	maior
<	menor
>=	maior ou igual
<=	menor ou igual

Vamos colocar a matemática e lógica - Operadores

Exemplos – Operadores Matemáticos:

```c
#include <stdio.h>

int main() {

    int a = 10;
    int b = 3;

    printf("Soma: %d\n", a + b);
    printf("Subtracao: %d\n", a - b);
    printf("Multiplicacao: %d\n", a * b);
    printf("Divisao inteira: %d\n", a / b);
    printf("Resto: %d\n", a % b);

    return 0;
}
```

E se eu tiver dois tipos diferentes? Qual especificador utilizar?

```c
#include <stdio.h>

int main() {
    int numero_1 = 10; // Variável do tipo inteiro
    float numero_2 = 20.2; // Variável do tipo float

    printf("%d ou %f\n", numero_1 / numero_2);

    return 0;
}
```

R: Sempre o maior, neste caso o float. Então será o especificador %f.

Vamos colocar a matemática e lógica - Operadores

Exemplos – Operadores Matemáticos:

E se eu tiver dois inteiros, porém o resultado deve ser decimal

```c
#include <stdio.h>

int main() {
    int numero_1 = 10; // Variável do tipo inteiro
    float numero_2 = 3; // Variável do tipo inteiro

    printf("%d\n", numero_1 / numero_2);

    return 0;
}
```

R: -1431655765



Solução:

```c
#include <stdio.h>

int main() {
    int numero_1 = 10; // Variável do tipo inteiro
    float numero_2 = 3; // Variável do tipo inteiro

    printf("%f\n", (float)numero_1 / numero_2);

    return 0;
}
```

Colocar o tipo na frente da operação, chamamos de **cast**

Inserir o especificador correto

Vamos colocar a matemática e lógica - Operadores

Exemplos – Operadores Relacionais:

```c
#include <stdio.h>

int main() {

    int a = 10;
    int b = 5;

    printf("%d\n", a > b);
    printf("%d\n", a == b);

    return 0;
}
```

R:

```
1
0
```

1 ← Verdadeiro

0 ← Falso

Importante!

a = b é diferente de a == b

Matemático

Relacional

Vamos colocar a matemática e lógica - Expressões

Uma expressão é qualquer combinação de:

- Valores (constantes); → São números ou caracteres fixos.
- Variáveis; → São espaços na memória que guardam valores.
- Operadores. → São os símbolos que fazem operações.

Que resulta em um valor final.

Exemplo:

```
int a = 5;
int resultado = a + 10;
```

Aqui temos:
- variável → a;
- valor → 10;
- operador → +.

👉 Isso é uma **expressão**.

Como o computador resolve uma expressão?



Vamos colocar a matemática e lógica - Expressões

Ordem de precedência

O computador segue regras:

```
int a = 5;
int b = 3;

int resultado = (a + b) > 10;
```

1. Parênteses ()
2. Multiplicação e divisão (*, /)
3. Soma e subtração (+, -)
4. Comparações (>, <, ==).

Expressões são o coração da programação. Tudo que um programa faz é calcular expressões e tomar decisões com base nelas.

Vamos praticar! Siga as instruções:

Para cada exercício:

Análise operacional

- Resolva passo a passo como o computador faria;
- Mostre cada etapa do cálculo.

Resultado final

- Escreva o valor final da variável.

Análise lógica

- Explique o que esse resultado significa.

Exercício exemplo:

```
int x = 2 + 3 * 4;
```

Esperado de você:

Operacional:

```
3 * 4 = 12

2 + 12 = 14
```

Resultado:

```
x = 14
```

Lógico:

```
Multiplicação acontece antes da soma.
```

Vamos praticar! Siga as instruções:

Exercício 1

int x = (2 + 3) * 4;

Exercício 2

int x = (2 + 3) * 4;

Exercício 3

float x = 10 / 3;

Exercício 4

float x = 10 / 3.0;

Exercício 5

float x = 10 / 3.0;

Exercício 6

int x = 5 + 7 / 2;

Exercício 7 (Relacional)

int x = 10 > 5;

Exercício 8

int x = 10 == 5;

Exercício 9

int x = (2 + 3) > (4 * 2);

Exercício 10 (Expressão mista)

int x = (10 / 2) + (3 > 1);

Exercício 11 (Muito importante)

int x = (5 > 3) * 10;

Exercício 12 (Nível mais alto)

int x = (8 + 2) / (3 + 2);

Exercício 14

int x = (8 + 2) / 3 + 2;

Exercício 15

int x = 5 / 2 * 2;

Exercício 16

int x = (10 > 5) + (3 * 2);

Vamos praticar! Siga as instruções:

Exercício 17

```
int a, b;
scanf("%d %d", &a, &b);
int x = a + b * 2;
```

👉 Pergunta:

1. Faça análise operacional
2. Se a = 3 e b = 4, qual o resultado?

Exercício 18

```
int idade;
scanf("%d", &idade);
int maior = idade >= 18;
```

👉 Pergunta:

Se idade = 20 → resultado?

Se idade = 15 → resultado?


DÚVIDAS?!
OBRIGADO!!

Copyright © 2026
Prof. Dr. Erick T. Yamamoto.
Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor (autor).